UNIT I – Problem Understanding and Representation

1. Problem Identification and Framing
2. Requirements and Expected Outcomes
3. Constraints and Assumptions
4. Problem Decomposition
5. Abstraction and Relevant Information
6. Structured Problem Representation

This unit explains how to understand a problem clearly, identify what needs to be solved, determine requirements and limitations, divide a complex problem into smaller parts, identify relevant information, and represent the problem in a structured way.

Learning objective:

By the end of this unit, students should be able to understand a problem before attempting to solve it, especially when developing algorithms, programs, or engineering solutions.

1. Problem Identification and Framing

1.1 What is a problem?

A problem is a situation in which we want to achieve a goal, but the method to achieve that goal is not immediately clear.

Simple examples

- A student wants to improve examination marks but does not know how to plan their study.
- A college wants to reduce canteen waiting time.
- A program needs to find the largest number in an array.
- A farmer wants to reduce water usage in agriculture.

In each case, there is a current situation and a desired outcome.

1.2 What is problem identification?

Problem identification is the process of recognizing and understanding what is wrong, missing, inefficient, or needs improvement.

Example: College Canteen

Current situation:

- Students wait a long time to purchase food.
- Several students arrive during the lunch break.
- The available counters may not serve everyone quickly.

Identified problem:

"Students experience long waiting times in the college canteen during lunch hours."

Notice that the problem is stated clearly without immediately assuming that adding more counters is the only solution.

1.3 What is problem framing?

Problem framing means expressing the identified problem in a clear and focused way so that the correct problem is addressed.

A poorly framed problem can lead to an ineffective solution.

Poor problem framing

"The canteen needs more counters."

This assumes a solution before investigating the cause.

Better problem framing

"Students experience long waiting times during lunch because the ordering and serving process may not handle peak demand efficiently."

This identifies the problem and leaves room to investigate its causes.

1.4 Elements of a good problem statement

A good problem statement should explain:

1. Current situation: What is happening now?
2. Problem: What is wrong or needs improvement?
3. Affected people or systems: Who experiences the problem?
4. Desired outcome: What should change?
5. Scope: Which part of the problem will be addressed?

Example: Student Marks

Problem statement:

"Students in a class are scoring below the expected level in mathematics. The objective is to identify the main learning difficulties and develop a study-support approach to improve their understanding."

This statement explains the current situation, problem, and objective without assuming a single cause.

1.5 Difference between a problem and a solution

Problem	Possible solution
Students wait too long in the canteen	Online ordering or improved service
Students cannot find books easily	Library search system
Electricity is wasted in empty classrooms	Automatic lighting controls
Students struggle to track assignments	Assignment management application

Important rule: First understand the problem; then identify possible solutions.

1.6 Steps for problem identification and framing

1. Observe the situation.
2. Identify the gap between the current and desired situations.
3. Collect relevant information.
4. Identify who is affected.
5. Investigate possible causes.
6. Write a clear problem statement.
7. Confirm that the statement addresses the actual problem.

Programming example

Problem: Find the largest number in an array.

- Current situation: An array contains several numbers.
- Goal: Identify the largest number.
- Input: An array of numbers.
- Output: The maximum value.
- Success condition: The output must be greater than or equal to every array element.

The problem is now clear enough to develop an algorithm.

2. Requirements and Expected Outcomes

2.1 What are requirements?

Requirements describe what a solution must do, provide, or satisfy.

They help us understand what is necessary before designing a solution.

Example: Online Library System

Possible requirements:

- Students must be able to search for books.
- Students must be able to check book availability.
- Librarians must be able to add and update books.
- The system must protect user information.

If requirements are incomplete, the final system may not satisfy users' needs.

2.2 Types of requirements

A. Functional requirements

Functional requirements describe what a system should do.

Examples:

- Allow a student to log in.
- Search for a book.
- Calculate student attendance percentage.
- Generate examination results.
- Display the largest number in an array.

B. Non-functional requirements

Non-functional requirements describe how well a system should work or the conditions under which it must operate.

Examples:

- The system should respond quickly.
- User information must be protected.
- The application should be easy to use.
- The system should work reliably.
- The application should support many users.

Comparison

Functional requirement	Non-functional requirement
What the system does	How well the system works
Calculate attendance	Calculate it accurately and efficiently
Search for a book	Return results quickly
Log in to the system	Protect login credentials

2.3 What are expected outcomes?

An expected outcome is the result that should be achieved after solving the problem.

A good expected outcome should be clear and, when possible, measurable.

Example: Reducing Canteen Waiting Time

Expected outcomes might include:

- Reduce average waiting time.
- Reduce ordering errors.
- Improve student satisfaction.
- Maintain affordable operating costs.

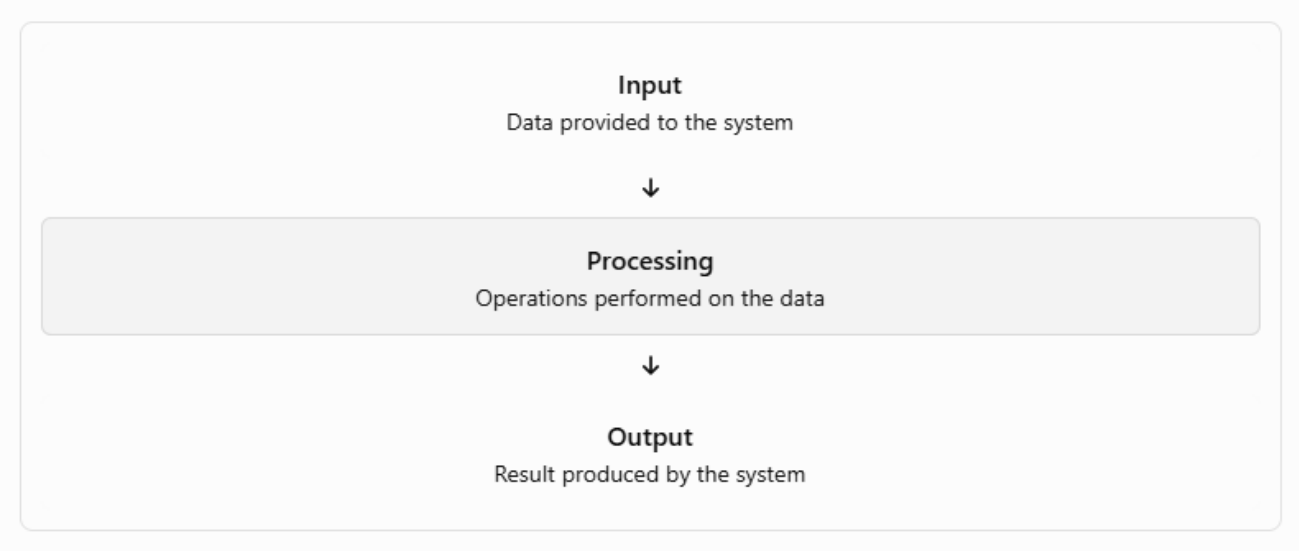
Suppose the current average waiting time is 12 minutes and the target is 6 minutes.

The target is measurable: reduce average waiting time by 50%.

These are illustrative targets; actual targets should be based on collected data.

2.4 Inputs, processing, and outputs

Many computational problems can be represented using the following structure:



Example: Calculate the average of three marks

- Input: 70, 80, 90
- Processing: Add the marks and divide by 3.
- Output: 80

Expected outcome: The program should calculate and display the correct average.

2.5 How do we check whether requirements are satisfied?

We define success criteria.

Examples:

- The program returns the correct answer.
- The application meets the required response time.
- The system prevents unauthorized access.
- The solution stays within the budget.
- Users can complete the intended task successfully.

Key idea: Requirements describe what is needed; expected outcomes describe what we want to achieve; success criteria explain how we will verify achievement.

3. Constraints and Assumptions

3.1 What are constraints?

A constraint is a limitation or restriction that a solution must satisfy.

Constraints affect which solutions are possible or acceptable.

Common types of constraints

Constraint	Meaning	Example
Time	Limited time available	Finish the project in 2 weeks
Budget	Limited money	Develop an app within ₹10,000
Memory	Limited storage or memory	Program must use limited RAM
Hardware	Available equipment	Only 10 computers are available
Technology	Technical limitations	Internet access is unavailable
Security	Required protection	Only authorized users can access records
Resources	Limited people or materials	Only 3 students are available

Example: College Event Application

A team wants to develop an event registration application.

Constraints:

- Development must finish within 15 days.
- Only three students are available.
- The budget is limited.
- The application must protect student information.

A solution requiring expensive hardware and several months of development may not be appropriate.

3.2 What are assumptions?

An assumption is something we accept as true for the purpose of analysing or solving a problem, even though it may not have been fully verified.

Assumptions help us proceed when complete information is unavailable.

Example: College Attendance

Suppose we are calculating attendance percentage.

We may assume:

- All classes have equal weight.
- The attendance records are accurate.

- The total number of classes is known.

These assumptions simplify the calculation. However, if some classes have different rules or the records are incorrect, the result may not reflect the actual attendance.

3.3 Difference between constraints and assumptions

Constraints	Assumptions
Limit what the solution can do	State what we temporarily accept as true
Usually imposed by the problem or environment	Used to simplify analysis or handle missing information
Example: Budget is ₹10,000	Example: Internet is available
Must be respected or addressed	Should be checked when possible

3.4 Boundary conditions

A boundary condition describes a special case at the edge of the allowed input or operating range.

Boundary conditions are important because solutions that work for normal cases may fail at the limits.

Example 1: Voting Eligibility

Suppose the rule is that a person can vote at age 18 or above.

- Age 17: Not eligible.
- Age 18: Eligible.
- Age 19: Eligible.

The age of exactly 18 is an important boundary case.

Example 2: Array

Suppose an array contains N elements.

- Empty array: N=0
- One element: N=1
- Multiple elements: N>1

An algorithm that finds the largest element must handle an empty array appropriately rather than assuming an element exists.

3.5 Why must we identify constraints and assumptions?

They help us:

1. Avoid unrealistic solutions.
2. Estimate required time and resources.
3. Identify missing information.
4. Make decisions logically.
5. Handle special cases.
6. Reduce unexpected failures.

Practice example

A college wants to build a student-result portal.

Identify:

- Two constraints
- Two assumptions
- Two boundary conditions

Possible considerations include development time, budget, availability of student records, zero students in a class, and a student with marks at the minimum or maximum allowed value.

4. Problem Decomposition

4.1 What is problem decomposition?

Problem decomposition is the process of breaking a large or complex problem into smaller, manageable subproblems.

A complex problem may be difficult to understand and solve as one large task. Dividing it into smaller tasks makes the work easier to plan, implement, test, and maintain.

Example: College Management System

Developing an entire college management system is a large task.

We can divide it into:

- Student registration
- Faculty management
- Attendance management
- Examination management
- Fee management

- Timetable management
- Report generation

Each module can be designed and tested separately.

4.2 Why is decomposition useful?

1. Reduces complexity

Smaller tasks are easier to understand.

2. Improves planning

We can estimate the time and resources needed for each subproblem.

3. Supports teamwork

Different team members can work on different modules.

4. Makes testing easier

Each component can be tested independently.

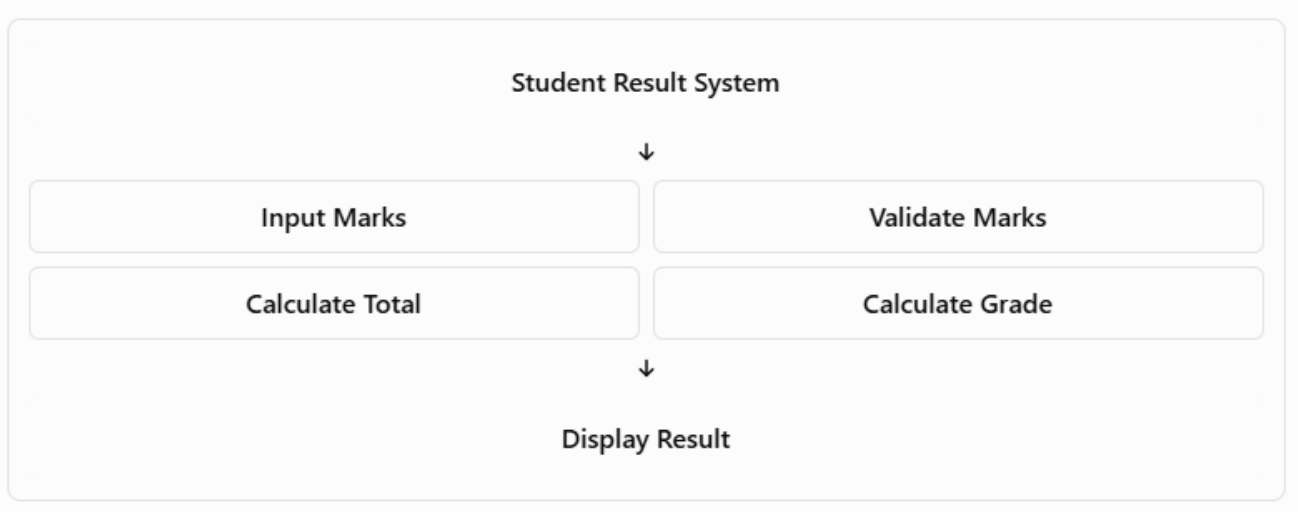
5. Simplifies debugging

Errors can be located within a smaller part of the system.

6. Encourages reuse

A useful component may be reused in another project.

4.3 Decomposition example: Student Result System



Each subproblem has a specific responsibility.

For example, the validation module checks whether marks are within the permitted range before the calculation module processes them.

4.4 Decomposition in programming

Consider the problem of calculating the average of five subjects and determining the result.

We can divide it into:

1. Read five marks.
2. Validate the marks.
3. Calculate the total.
4. Calculate the average.
5. Determine the result or grade according to the given rules.
6. Display the result.

Each step can be developed as a separate function if appropriate.

4.5 Top-down approach

The top-down approach starts with the complete problem and progressively breaks it into smaller subproblems.

Example:

Student Result System

```
|
|-- Read Marks
|-- Validate Marks
|-- Calculate Total
|-- Calculate Average
|-- Determine Grade
|-- Display Result
```

We start with the overall goal and move toward smaller tasks.

4.6 Important point: Decomposition and dependencies

Sometimes one subproblem depends on another.

For example:

- Marks must be entered before they can be validated.
- Marks must be validated before calculating the total.
- The total must be calculated before calculating the average.

Therefore, decomposition should identify not only the smaller tasks but also their order and dependencies.

5. Abstraction and Relevant Information

5.1 What is abstraction?

Abstraction means focusing on the important details of a problem while ignoring details that are not necessary for the current task.

It helps reduce complexity by concentrating on what matters.

Simple everyday example

When using an ATM, a customer generally sees options such as:

- Withdraw cash
- Check balance
- Change PIN

The customer does not need to understand every internal operation of the bank's servers to withdraw money.

The interface provides the relevant operations while hiding unnecessary internal details.

5.2 Example: Student Attendance System

Suppose we want to calculate a student's attendance percentage.

Relevant information:

- Classes attended
- Total classes conducted

For this calculation, the student's height, home address, favourite subject, and blood group are not required.

The relevant formula is:

The relevant formula is:

$$\text{Attendance Percentage} = \frac{\text{Classes Attended}}{\text{Total Classes}} \times 100$$

If a student attended 36 out of 40 classes:

$$\frac{36}{40} \times 100 = 90\%$$

The calculation focuses only on the information needed to solve the problem.

A complete attendance application may require additional information for other features, such as identifying the student or generating reports.

In C++, abstraction is implemented mainly using classes and access specifiers, and also through abstract classes and pure virtual functions.

5.3 What is relevant information?

Relevant information directly contributes to understanding the problem, making a decision, or producing the expected result.

Example: Selecting a Laptop for Programming

Relevant information may include:

- RAM
- Processor
- Storage
- Battery life
- Price
- Software compatibility

Less relevant information may include the colour of the packaging, unless appearance is an explicit purchasing requirement.

Relevance depends on the goal. A detail that is unnecessary for one problem may be important for another.

5.4 Abstraction versus ignoring information

Abstraction does not mean ignoring every detail that appears unimportant at first. It means identifying which details are necessary for the current purpose.

For example, in an attendance system:

- Student marks may be irrelevant to attendance calculation.
- Student identity is important when recording attendance.
- Privacy and access control are important when designing the complete system.

Good abstraction removes unnecessary complexity without losing information essential to correctness, safety, or the requirements.

5.5 Levels of abstraction

A complex system can be understood at different levels.

Example: Online Shopping System

Level	Focus
High-level	Customer buys a product
Middle-level	Search, cart, payment, order management
Low-level	Database queries, validation, network requests, algorithms

At a high level, we focus on the overall purpose. At a lower level, we examine implementation details.

5.6 Benefits of abstraction

- Reduces complexity
- Improves understanding
- Helps organize information
- Makes design easier
- Supports modular development
- Helps us focus on the actual objective

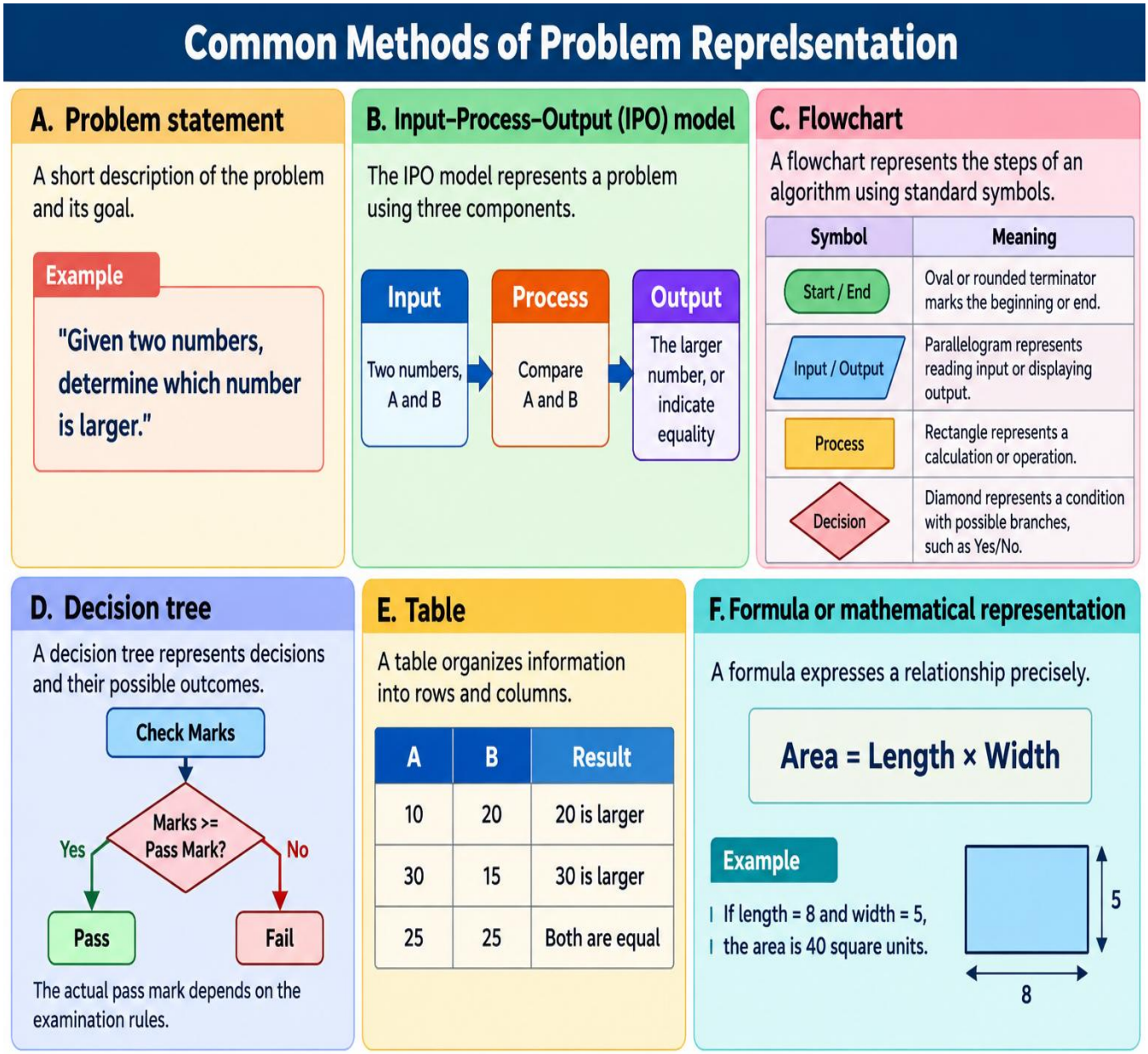
6. Structured Problem Representation

6.1 What is structured problem representation?

Structured problem representation means expressing a problem in an organized form so that it becomes easier to understand, analyse, communicate, and solve. Instead of keeping the entire problem in our mind, we represent its important elements using text, diagrams, tables, flowcharts, formulas, or models.

Why is it useful?

- Makes the problem easier to understand.
- Identifies inputs and outputs.
- Shows the steps required to solve the problem.
- Reveals relationships between components.
- Helps identify missing requirements.
- Makes it easier to communicate ideas to others.



6.3 Choosing an appropriate representation

Different representations serve different purposes.

Representation	Best used for
Problem statement	Clearly defining the problem
IPO model	Identifying inputs, processing, and outputs
Flowchart	Showing the sequence of steps
Decision tree	Showing decisions and outcomes
Table	Organizing or comparing information
Formula	Expressing mathematical relationships
Block diagram	Showing major components and their connections

A single problem may benefit from more than one representation.

7. Integrated Example – College Library Book Search

Let's apply all six Unit I topics to one real-world problem.

Problem

Students spend too much time searching for books in the college library.

1. Problem Identification and Framing

Students cannot easily locate the required books. The goal is to help them find books quickly.

2. Requirements and Expected Outcomes

Students should search by book title or author and view availability and shelf location. The expected outcome is faster book discovery.

3. Constraints and Assumptions

Constraints may include budget, available computers, and development time. An assumption might be that the library has a digital or digitizable book catalogue.

4. Problem Decomposition

Divide the system into book data management, search, availability checking, shelf-location display, and administration.

5. Abstraction and Relevant Information

Focus on book title, author, availability, and shelf location. Ignore unrelated information for the basic search operation.

6. Structured Problem Representation

Represent the system using an IPO model, a flowchart for searching, and tables for book records and search results.

The example demonstrates how understanding and representing the problem provides a foundation for designing a practical solution.

A useful sequence to remember is:

Identify → Define → Specify → Constrain → Decompose → Abstract → Represent → Solve → Verify

- Part A: 20 questions with answers
- Part B: 30 multiple-choice questions with answers
- Part C: 5 small classroom tasks
- Part D: 2 in-depth assignment tasks

Part A – 20 Questions and Answers

Topic 1: Problem Identification and Framing

1. What is a problem?

Answer: A problem is a situation in which we want to achieve a goal, but the method to achieve that goal is not immediately clear.

Example: A student wants to improve examination marks but does not know how to plan their study.

2. What is problem identification and problem framing?

Answer:

- Problem identification means recognizing what is wrong, missing, or needs improvement.
- Problem framing means expressing the problem clearly and focusing on what needs to be solved.

Example:

Poor framing: "The canteen needs more counters."

Better framing: "Students experience long waiting times in the canteen during lunch hours."

The better statement identifies the problem without assuming a solution.

Topic 2: Requirements and Expected Outcomes

3. What are requirements?

Answer: Requirements describe what a solution must do, provide, or satisfy.

Example: An online library system should allow students to search for books and check their availability.

4. Differentiate between functional and non-functional requirements.

Answer:

Functional requirements	Non-functional requirements
Describe what a system does	Describe how well a system works
Search for books	Return results quickly
Calculate attendance	Calculate it accurately
Generate examination results	Protect student information

5. What are expected outcomes and success criteria?

Answer:

- Expected outcomes are the results we want to achieve.
- Success criteria are the conditions used to verify whether those results have been achieved.

Example: The expected outcome is to reduce canteen waiting time from 12 minutes to 6 minutes. The success criterion is that the measured average waiting time reaches the target.

6. Explain the Input–Process–Output (IPO) model with an example.

Answer: The IPO model represents a problem through three components:

- Input: Data provided to the system.
- Process: Operations performed on the data.
- Output: Result produced by the system.

Example: Calculate the average of three marks.

- Input: 70, 80, 90
- Process: Add the marks and divide by 3.
- Output: 80

Topic 3: Constraints and Assumptions

7. What is a constraint? Give three examples.

Answer: A constraint is a limitation or restriction that a solution must satisfy.

Examples:

1. Time: Complete the project in two weeks.
2. Budget: Develop an application within ₹10,000.
3. Memory: A program must operate within a specified memory limit.

8. What is an assumption? Give an example.

Answer: An assumption is something accepted as true for solving a problem, even though it has not been fully verified.

Example: While calculating attendance percentage, we assume that all classes have equal weight.

9. Differentiate between constraints and assumptions.

Answer:

Constraints	Assumptions
Limit what a solution can do	State what we temporarily accept as true
Must be respected or addressed	Should be verified when possible
Example: Limited budget	Example: Internet access is available

10. What is a boundary condition? Why is it important?

Answer: A boundary condition describes a special case at the edge of the allowed input or operating range.

It is important because a solution that works for normal inputs may fail at the limits.

Example: If a person is eligible to vote at age 18 or above, age 18 is an important boundary case.

For an array, an empty array and a single-element array are also important boundary cases.

Topic 4: Problem Decomposition

11. What is problem decomposition?

Answer: Problem decomposition is the process of breaking a large or complex problem into smaller, manageable subproblems.

Example: A college management system can be divided into student registration, attendance, examinations, fee management, and report generation.

12. What are the benefits of problem decomposition?

Answer:

1. Reduces complexity.
2. Makes planning easier.
3. Supports teamwork.
4. Makes testing easier.
5. Simplifies debugging.
6. Encourages component reuse.

13. What is the top-down approach?

Answer: The top-down approach starts with the complete problem and progressively divides it into smaller subproblems.

Example:

Student Result System

|

|-- Read Marks

|-- Validate Marks

|-- Calculate Total

|-- Calculate Average

|-- Determine Grade

|-- Display Result

14. What are dependencies between subproblems?

Answer: Dependencies occur when one task must be completed before another task can be performed.

Example: In a student result system, marks must be entered before validation, and valid marks must be available before calculating the total.

Topic 5: Abstraction and Relevant Information

15. What is abstraction?

Answer: Abstraction means focusing on the important details of a problem while ignoring details that are not necessary for the current task.

Example: When using an ATM, a customer can withdraw money without understanding the internal operations of the bank's servers.

16. What is relevant information? Give an example.

Answer: Relevant information directly contributes to understanding a problem, making a decision, or producing the required result.

Example: To calculate attendance percentage, the relevant information is the number of classes attended and the total number of classes conducted.

A student's height and favourite colour are not required for this calculation.

17. Explain the different levels of abstraction with an example.

Answer: A system can be understood at different levels of detail.

Example: Online shopping system.

- High level: A customer buys a product.
- Middle level: Search, shopping cart, payment, and order management.
- Low level: Database queries, validation, network requests, and algorithms.

18. Does abstraction mean ignoring all details that appear unimportant?

Answer: No. Abstraction means ignoring only details that are unnecessary for the current purpose.

Information that is irrelevant to one task may be essential to another.

Example: Student marks may be irrelevant to attendance calculation, but student identity is important when recording attendance.

Topic 6: Structured Problem Representation

19. What is structured problem representation? Name four methods.

Answer: Structured problem representation means expressing a problem in an organized form so that it becomes easier to understand, analyse, communicate, and solve.

Four methods are:

1. Problem statement
2. Input–Process–Output model
3. Flowchart
4. Decision tree

Other methods include tables, formulas, and block diagrams.

20. Explain how to represent the problem of finding the largest of two numbers.

Answer:

- Problem statement: Given two numbers, determine which is larger.
- Input: Two numbers, A and B.
- Process: Compare A and B.
- Output: Display the larger number or indicate equality.

Pseudocode:

```
START
   Read A, B
   IF A > B
      PRINT "A is larger"
   ELSE IF B > A
      PRINT "B is larger"
   ELSE
      PRINT "Both are equal"
   END IF
STOP
```

The test cases should include A greater than B, B greater than A, and A equal to B.

Part B – 30 Multiple-Choice Questions (MCQs)

Topic 1: Problem Identification and Framing

1. What is a problem?

A. A situation with no goal B. A situation where the method to achieve a goal is not immediately clear C. A completed task D. A programming language

Answer: B

2. What should be done before selecting a solution?

A. Start coding B. Purchase equipment C. Understand and define the problem D. Select a programming language

Answer: C

3. Which is the best problem statement?

A. The canteen needs a new building. B. Students experience long waiting times in the canteen during lunch hours. C. Buy more computers. D. Develop an application immediately.

Answer: B

4. Which is an element of a good problem statement?

A. Current situation B. Desired outcome C. Scope of the problem D. All of the above

Answer: D

5. What is the main purpose of problem framing?

A. To start coding immediately B. To express the problem clearly and focus on what needs to be solved C. To ignore the causes D. To select the most expensive solution

Answer: B

Topic 2: Requirements and Expected Outcomes

6. Requirements describe:

A. Only the cost of a project B. What a solution must do, provide, or satisfy C. Only the programming language D. Only the project title

Answer: B

7. Which is a functional requirement?

A. The system should be secure. B. The system should be reliable. C. Students should be able to search for books. D. The system should respond quickly.

Answer: C

8. Which is a non-functional requirement?

A. Generate examination results B. Search for books C. Calculate attendance D. Protect user information

Answer: D

9. What is an expected outcome?

A. A limitation on the project B. A result that should be achieved C. An unverified assumption D. A programming error

Answer: B

10. What is the purpose of success criteria?

A. To verify whether the desired outcome has been achieved B. To increase the number of requirements C. To avoid testing D. To select a programming language

Answer: A

11. In the IPO model, which component performs operations on input data?

A. Input B. Process C. Output D. Constraint

Answer: B

Topic 3: Constraints and Assumptions

12. Which of the following is a constraint?

A. The project must be completed within two weeks. B. The internet is available. C. The input data is correct. D. All users have computers.

Answer: A

13. Which is an assumption?

A. The project budget is ₹10,000. B. The project deadline is Friday. C. The internet connection is available. D. Only five computers are available.

Answer: C

14. What is the main difference between a constraint and an assumption?

A. They have exactly the same meaning. B. A constraint limits a solution, while an assumption is accepted as true for analysis. C. An assumption always limits the budget. D. A constraint is always unverified.

Answer: B

15. If a person is eligible to vote at age 18 or above, which is an important boundary case?

A. Age 10 B. Age 15 C. Age 18 D. Age 30 only

Answer: C

16. Which is an important boundary case for an algorithm that finds the largest element in an array?

A. Empty array B. Single-element array C. Array with multiple elements D. All of the above

Answer: D

Topic 4: Problem Decomposition

17. Problem decomposition means:

A. Combining all tasks into one large task B. Breaking a complex problem into smaller subproblems C. Ignoring difficult tasks D. Removing all requirements

Answer: B

18. Which is a benefit of problem decomposition?

A. It makes testing more difficult. B. It prevents teamwork. C. It makes complex problems easier to manage. D. It eliminates the need for planning.

Answer: C

19. The top-down approach starts with:

A. The smallest task B. The complete problem C. The final output only D. A random subproblem

Answer: B

20. In a student result system, which task should normally happen before calculating the average?

A. Display the final result B. Validate the marks and calculate the total C. Delete the marks D. Print the grade before processing

Answer: B

Topic 5: Abstraction and Relevant Information

21. What is abstraction?

A. Including every available detail B. Focusing on essential details and ignoring unnecessary ones C. Removing all information D. Making a problem more complex

Answer: B

22. Which information is relevant when calculating attendance percentage?

A. Student's favourite colour B. Student's height C. Classes attended and total classes D. Student's favourite sport

Answer: C

23. When using an ATM, which is an example of abstraction?

A. The user must understand every bank server operation. B. The user can withdraw cash without knowing the internal implementation. C. The user must write a program. D. The user must know the bank's database design.

Answer: B

24. Which statement about abstraction is correct?

A. All details must always be ignored. B. Only the appearance of a system matters. C. Relevant details depend on the purpose of the problem. D. Abstraction prevents problem solving.

Answer: C

25. In an online shopping system, which is a high-level view?

A. A customer buys a product. B. A database query executes. C. A network request is processed. D. A variable is declared.

Answer: A

Topic 6: Structured Problem Representation

26. Which representation is best for identifying inputs, processing, and outputs?

A. IPO model B. Decision tree only C. Paragraph of unrelated text D. Project budget

Answer: A

27. Which representation uses standard symbols to show the steps of an algorithm?

A. Table B. Flowchart C. Assumption D. Requirement list

Answer: B

28. Which symbol is commonly used for a decision in a flowchart?

A. Rectangle B. Oval C. Diamond D. Parallelogram

Answer: C

29. Which representation is most suitable for organizing values and their corresponding results?

A. Table B. Assumption C. Problem statement only D. Constraint list

Answer: A

30. Which sequence best represents the overall problem-solving process taught in Unit I?

A. Code → Submit → Stop B. Identify → Define → Specify → Constrain → Decompose → Abstract → Represent → Solve → Verify C. Guess → Build → Ignore → Finish D. Select → Code → Avoid testing

Answer: B

Part C – 5 Small Classroom Tasks

These tasks are suitable for individual or pair work. Students should write their reasoning clearly rather than provide only the final answer.

Task 1 – Frame the Problem

Scenario: Students complain that they cannot find the books they need in the college library.

Complete the following:

1. Write the current situation.
2. Identify the main problem.
3. Identify the people affected.
4. State the desired outcome.
5. Write a clear problem statement.
6. Suggest two possible solutions without assuming that either is automatically the best.

Topics covered: Problem identification and framing.

Task 2 – Identify Requirements and Constraints

Scenario: Your college wants an online assignment submission system.

Write:

1. Five functional requirements.
2. Three non-functional requirements.
3. Three constraints.
4. Two assumptions.
5. Three measurable success criteria.

Topics covered: Requirements, expected outcomes, constraints, and assumptions.

Task 3 – Decompose a Problem

Scenario: Develop a student result system that accepts five subject marks and displays the total, average, and grade.

Your tasks:

1. Divide the problem into at least six smaller steps.
2. Arrange the steps in the correct order.
3. Identify which steps depend on earlier steps.
4. Identify two tasks that could be developed as separate functions.
5. Explain how decomposition makes testing easier.

Topics covered: Problem decomposition and dependencies.

Task 4 – Identify Relevant Information

Scenario: A student wants to choose a laptop for programming.

Consider the following information:

- RAM
- Processor
- Storage
- Price
- Battery life
- Favourite colour
- Packaging design
- Software compatibility
- Warranty
- Brand logo size

Your tasks:

1. Select the most relevant factors for the decision.
2. Identify the factors that are less relevant to programming requirements.
3. Explain why relevance depends on the goal.
4. Identify any important information that is missing.

Topics covered: Abstraction and relevant information.

Task 5 – Represent a Problem

Scenario: Given two numbers, determine the larger number or report that both are equal.

Your tasks:

1. Write a problem statement.
2. Identify the inputs, processing, and outputs.
3. Create a comparison table containing at least four test cases.
4. Draw a flowchart.
5. Write pseudocode.
6. Include a test case where both numbers are equal.

Topics covered: Structured problem representation and boundary cases.

Part D – 2 In-Depth Assignment Tasks

Assignment 1 – Smart College Library Search System

Topics: All six Unit I topics

Problem statement

Students spend too much time finding books in the college library. The college wants to develop a system that allows students to search for books, check availability, and identify shelf locations.

Assignment requirements

Task 1: Problem identification and framing

- Describe the current situation.
- Identify the problem and affected stakeholders.
- Write a clear problem statement.
- Define the scope of the proposed solution.

Task 2: Requirements and expected outcomes

- Identify at least six functional requirements.
- Identify at least four non-functional requirements.
- Define five expected outcomes or success criteria.

Task 3: Constraints and assumptions

- Identify at least four constraints.
- Identify at least three assumptions.
- Identify at least three boundary or exceptional cases.

Task 4: Problem decomposition

- Divide the system into at least six modules.
- Explain the purpose of each module.

- Identify dependencies between modules.
- Draw a top-down decomposition diagram.

Task 5: Abstraction and relevant information

- Identify the information required for searching books.
- Identify information that is unnecessary for the basic search operation.
- Explain which information may become relevant to other library functions.

Task 6: Structured problem representation Prepare the following:

1. Problem statement
2. Input–Process–Output model
3. Flowchart for searching a book
4. Table containing sample book records
5. Decision tree for checking book availability
6. A simple representation of the overall system

Submission format

- Written report: 5–8 pages
- Diagrams: At least three
- Presentation: 5 minutes
- Include a conclusion explaining how clear problem understanding can prevent design mistakes.

Assignment 2 – Student Result Analysis System

Topics: Requirements, constraints, decomposition, abstraction, and representation

Problem statement

A college currently calculates student results manually. The process takes time and may produce calculation errors. Design a structured solution that accepts subject marks, validates them, calculates totals and averages, determines grades according to supplied rules, and displays results.

Assignment requirements

Task 1: Define the problem

- Write a clear problem statement.
- Identify inputs, outputs, and expected outcomes.
- Specify the scope of the system.

Task 2: Define the rules

Use the following illustrative rules:

- The student has five subjects.
- Each subject is marked out of 100.
- Valid marks range from 0 to 100, inclusive.
- Total marks are the sum of the five subject marks.
- Average marks equal total marks divided by five.

For the assignment, define a simple grading scheme and state it explicitly. Do not assume it is an official college grading policy.

Task 3: Identify constraints and assumptions

- List at least four constraints.
- List at least three assumptions.
- Explain how invalid marks should be handled.
- Consider missing marks and the minimum and maximum valid marks.

Task 4: Decompose the problem Divide the system into smaller tasks, such as:

- Read student details.
- Read subject marks.
- Validate marks.
- Calculate total.
- Calculate average.
- Determine grade.
- Display results.

Explain the dependencies between these tasks.

Task 5: Apply abstraction

- Identify information essential for calculating the result.
- Identify information needed only for student identification or reporting.
- Identify information unnecessary for the calculation itself.

Task 6: Structured representation Prepare:

1. IPO model
2. Flowchart
3. Pseudocode
4. Sample result table
5. Test cases

6. A decomposition diagram

Required test cases

Include at least these situations:

- All subjects have high marks.
- All subjects have zero marks.
- One subject has the maximum mark of 100.
- One subject has an invalid mark, such as 105.
- One subject has a negative mark.
- All five marks are valid but different.